# 搜索形式化与搜索求解

祝恩
计算机学院
国防科技大学

Contents

# 学习内容

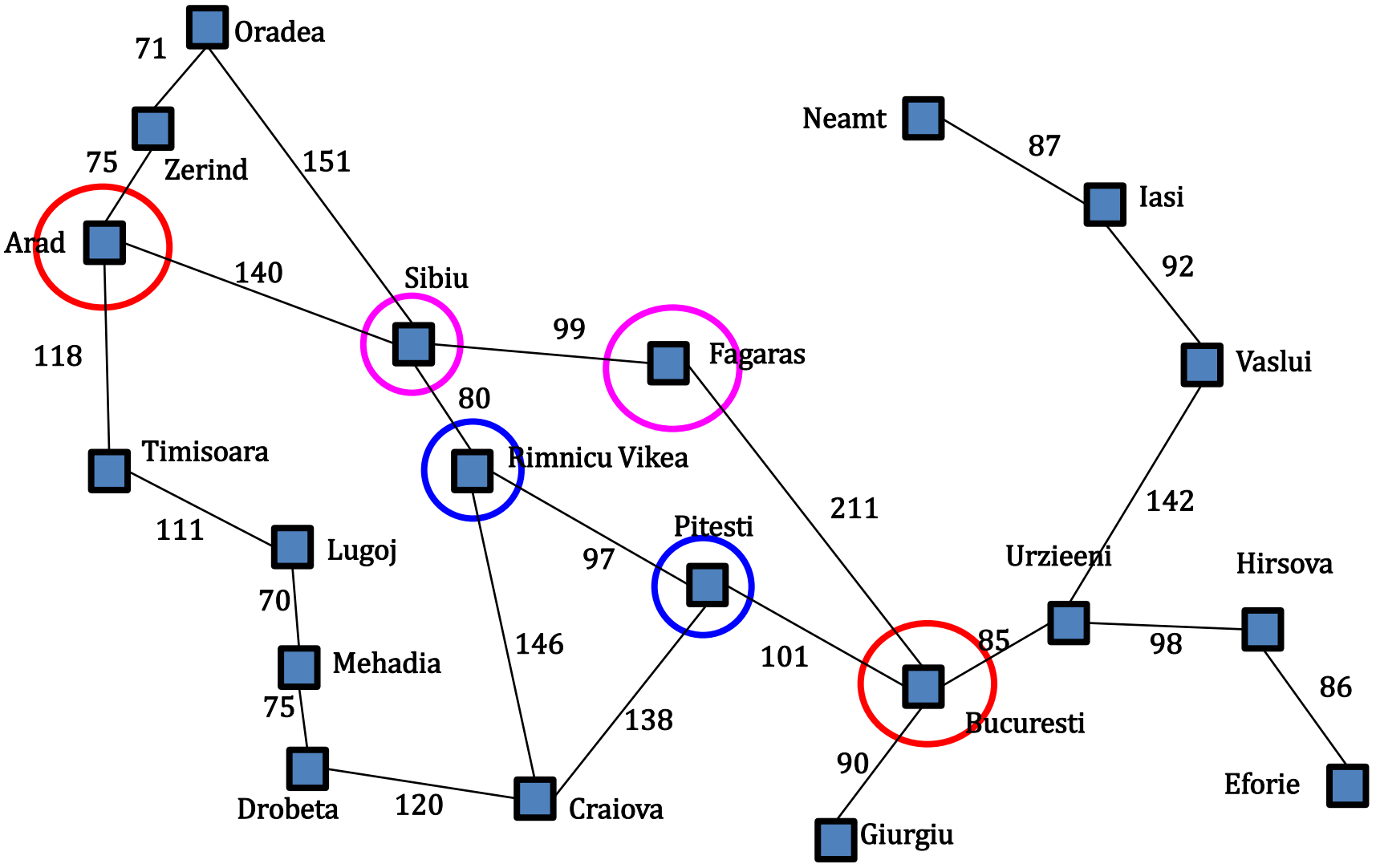
## 搜索的形式化与求解:

1. 搜索问题的形式化
2. 搜索求解
   树搜索, 图搜索, 树结点, 算法评估
3. 搜索策略

形式化“给定初始状态，如何前往目标状态”的问题

# 1搜索问题的形式化

# Romania问题

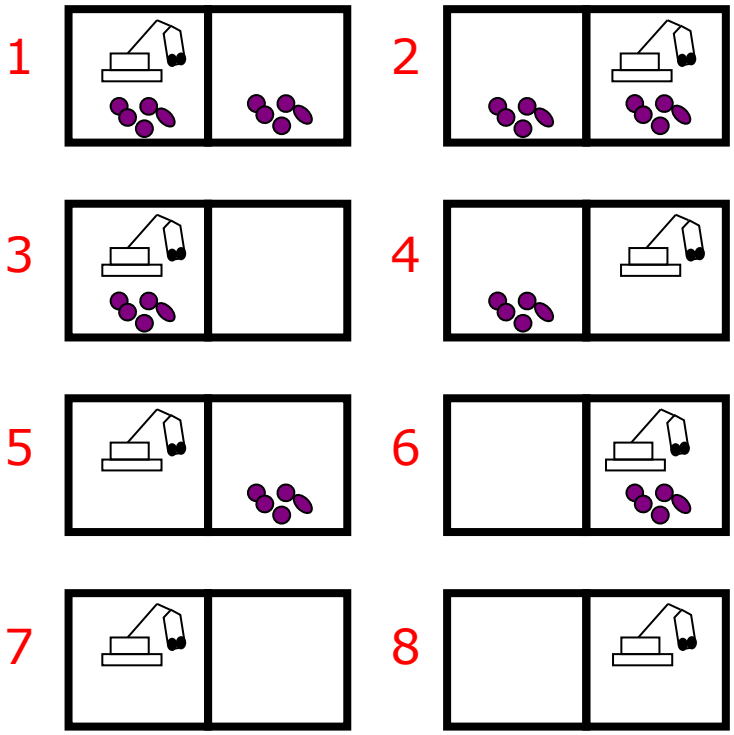

Oradea
71
Neamt
87
Zerind
75
151
Iasi
Arad
140
92
Sibiu
99
Fagaras
Vaslui
118
80
Timisoara
Rimnicu Vikea
142
211
111
Pitesti
Urzieeni
Hirsova
Lugoj
97
70
146
85
98
Mehadia
101
86
75
138
Bucuresti
90
Eforie
Drobeta
120
Craiova
Giurgiu

# 搜索问题形式化

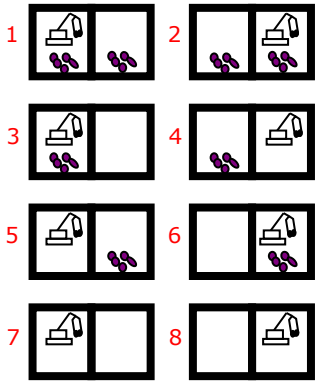
- a problem can be defined formally by five components:
  1. initial state — In(Arad)
  2. Actions(s): — Available actions in s
  3. Result(s,a): — transition model
  4. Goal-test(s) — {In(Bucuresti)}
  5. Cost(s,a): — action cost, length in kilometers
- a solution to a problem is an action sequence that leads from the initial state to a goal state.

# 实例：吸尘器问题

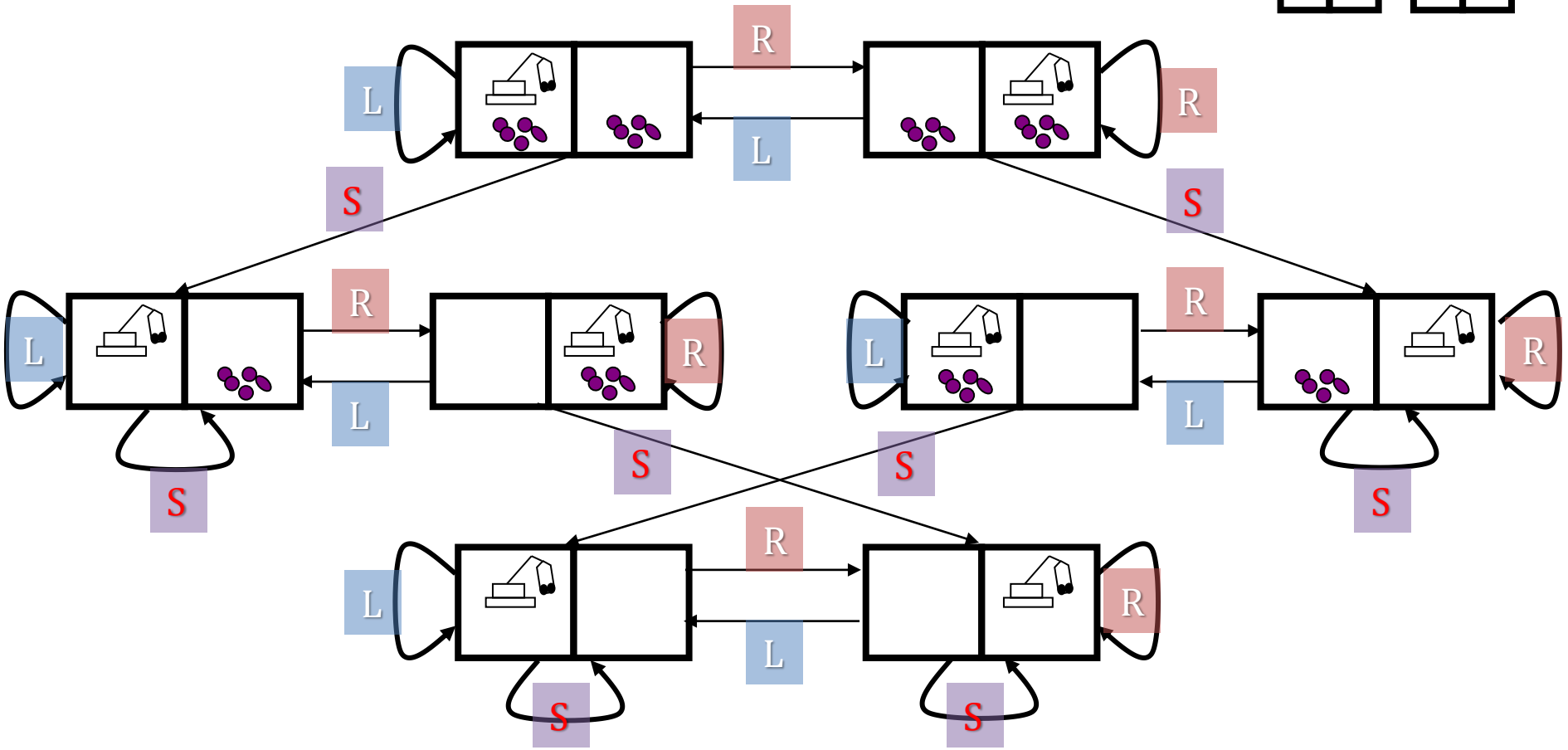
- States
- Initial State
  - 5
- Actions
  - Right, Left, Suck
- transition model
  - Result(1,Suck)=5
- Goal
  - {7,8}
  - Goal Test
- Action Cost
- Solution
  - [Right, Suck]



# 简单问题可表示为图




- Represent problem as a graph
  - Nodes are states
  - Arcs are actions



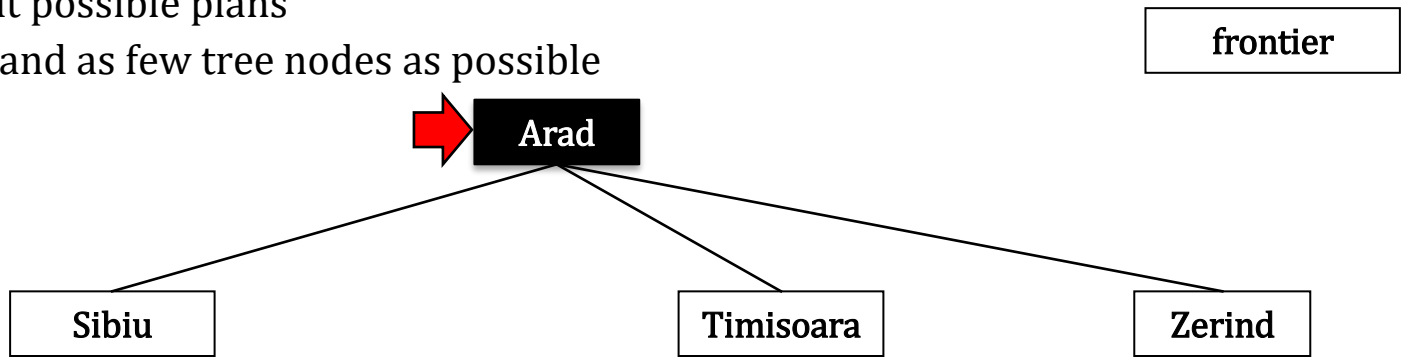
“搜索的本质”是什么?

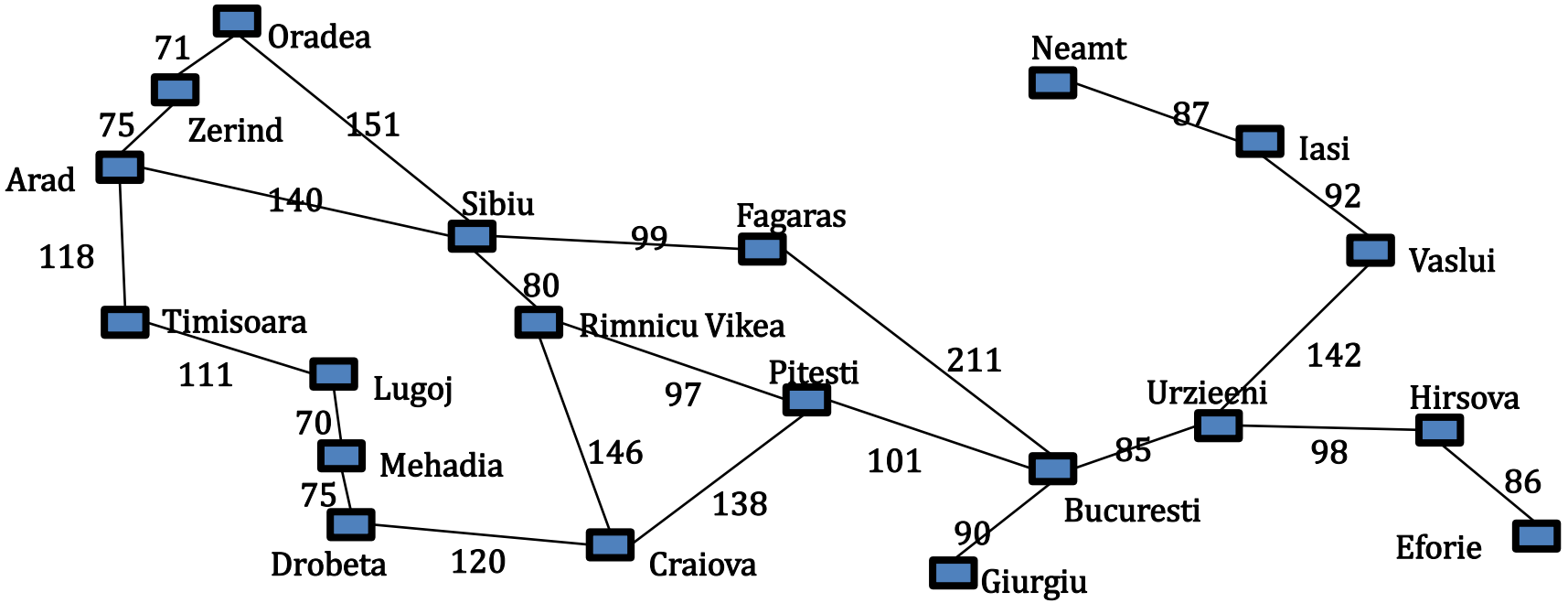
# 2 搜索求解:

## 树搜索,图搜索,树结点,算法评估

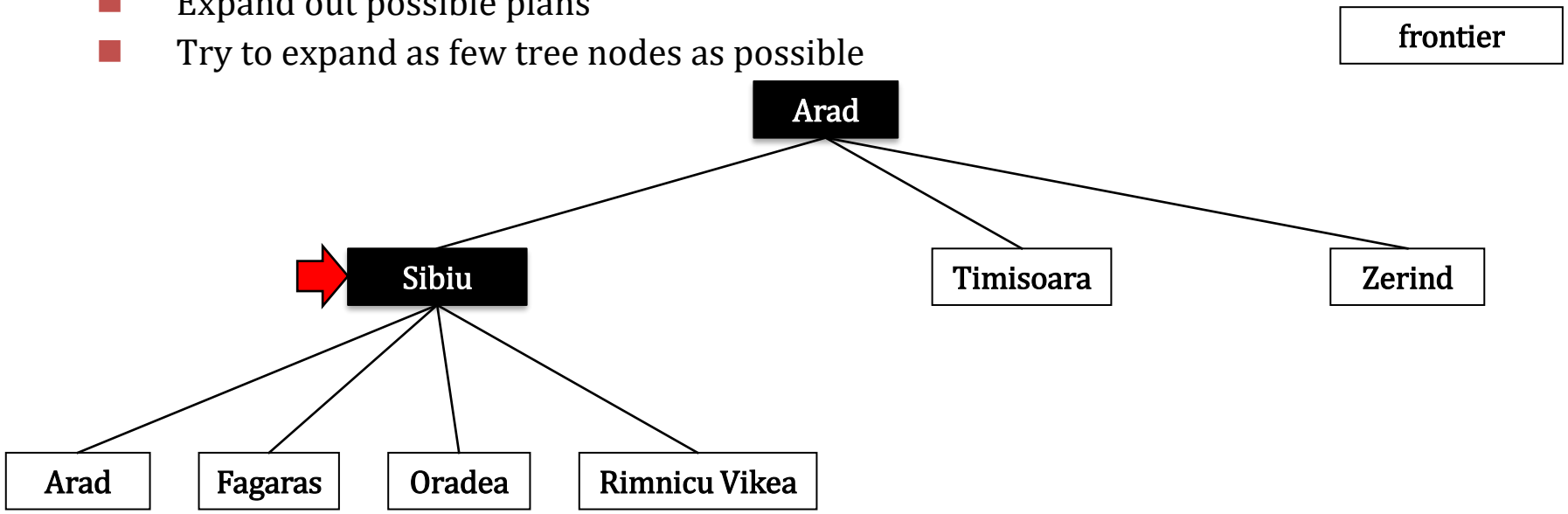
# (1)树搜索(TREE SEARCH)

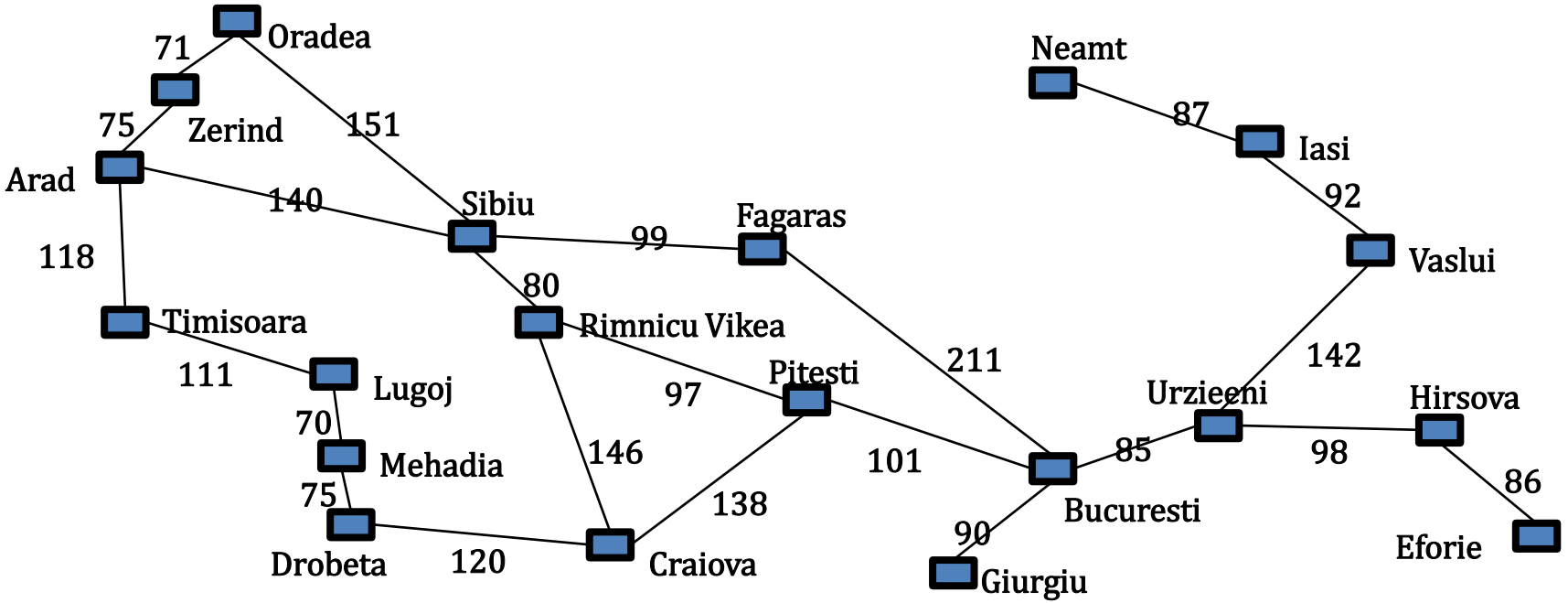
- Search
  - Expand out possible plans
  - Try to expand as few tree nodes as possible



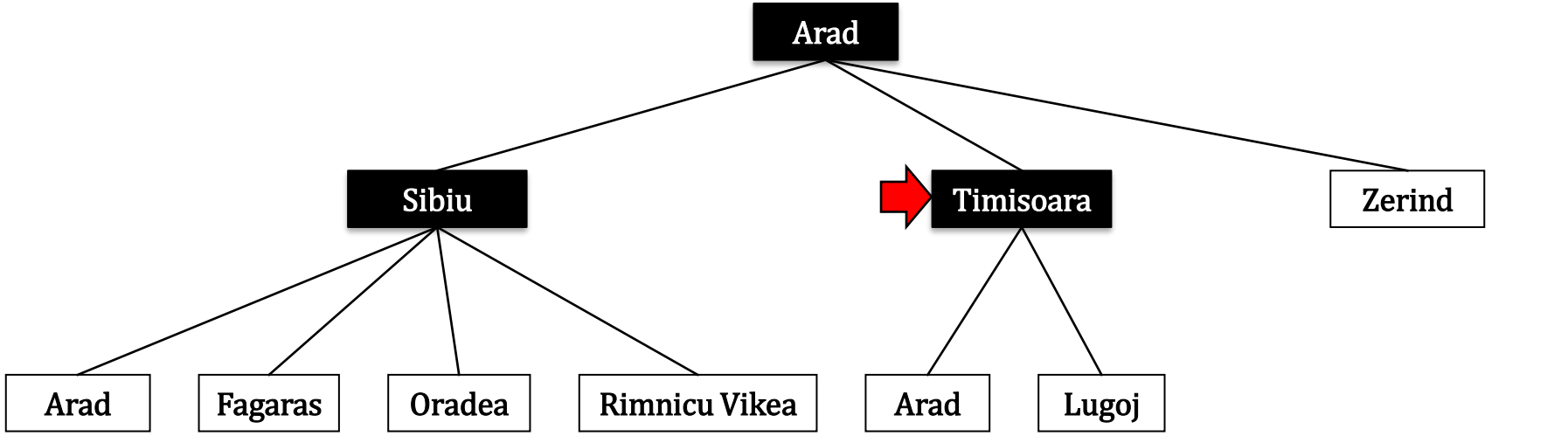



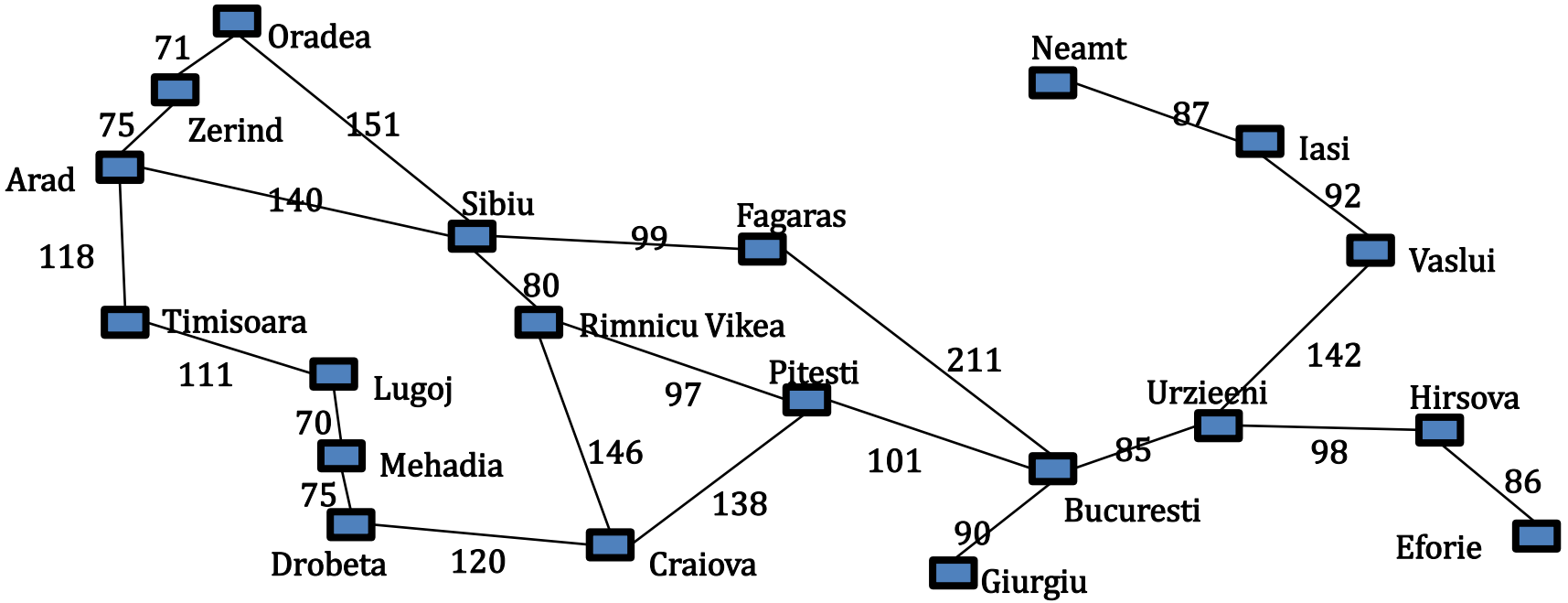
- Search
  - Expand out possible plans
  - Try to expand as few tree nodes as possible



- Search
  - Expand out possible plans
  - Try to expand as few tree nodes as possible



# 树搜索（Tree Search）

- Basic solution method for graph problems
  - Offline simulated exploration of state space
  - Searching a model of the space, not the real world

```
function Tree-Search(problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        expand the chosen node, adding the resulting nodes to the frontier
```

after it is removed from the frontier

# (2)图搜索(GRAPH SEARCH)

# 图搜索（Graph search）

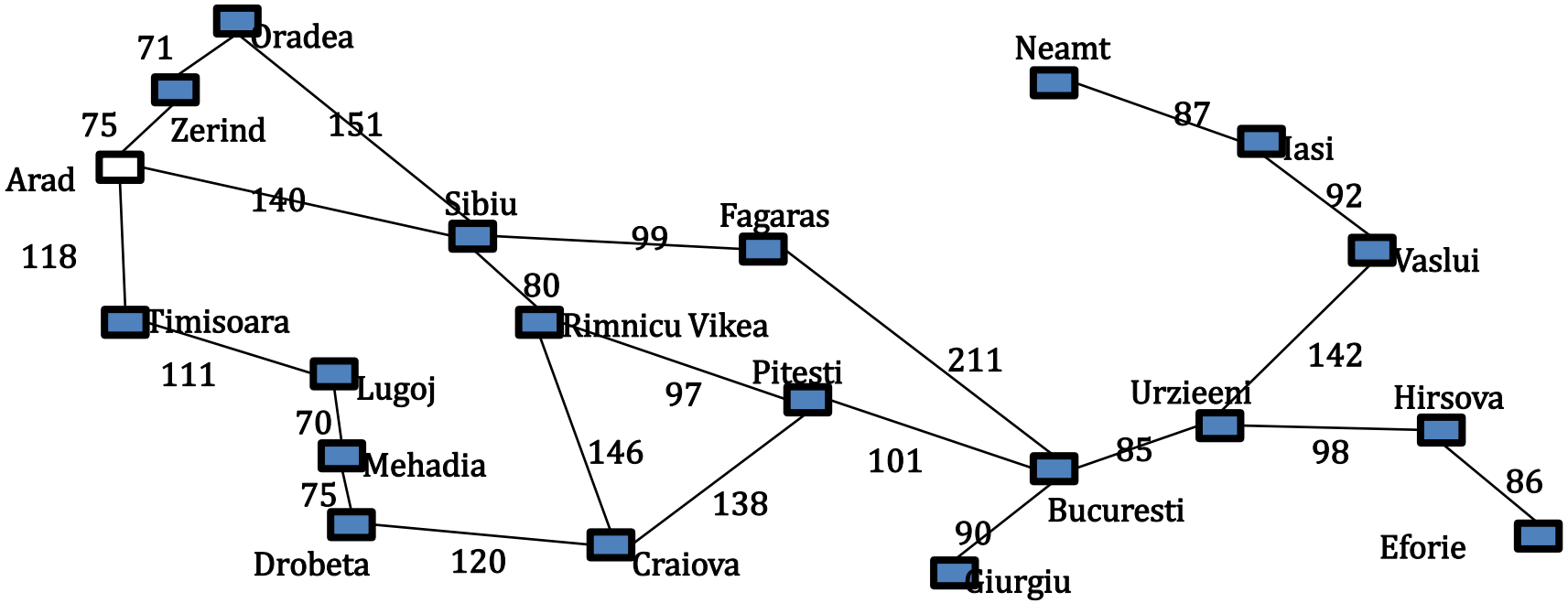
**function** Graph-Search(problem) **returns** a solution, or failure
  initialize the frontier using the initial state of problem
  *initialize the explored set to be empty*
  **loop do**
    **if** the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    **if** the node contains a goal state **then return** the corresponding solution
    *add the node to the explored set*
    expand the chosen node, adding the resulting nodes to the frontier
      *only if not in the frontier or explored set*

after it is removed from the frontier

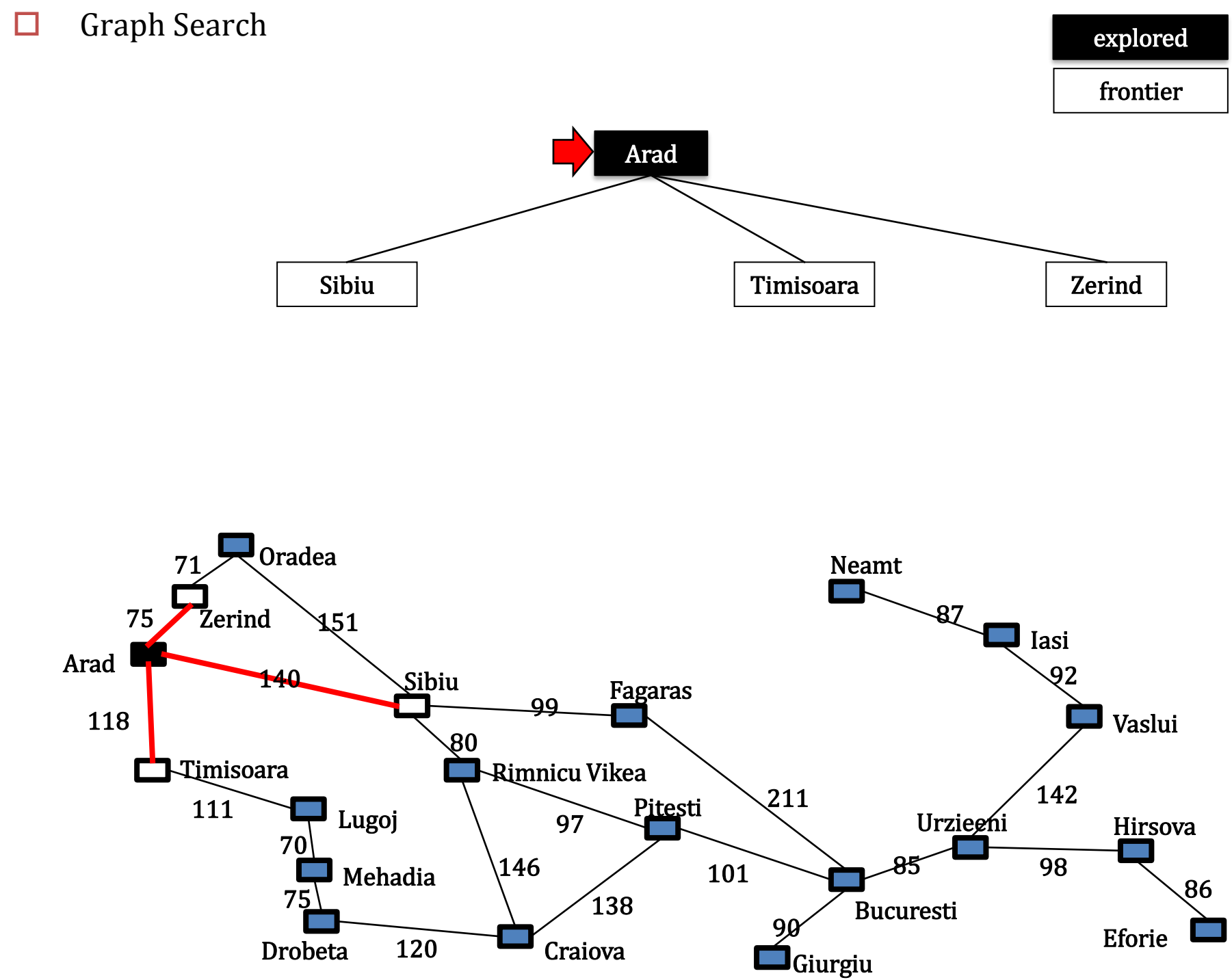
# Graph Search

frontier

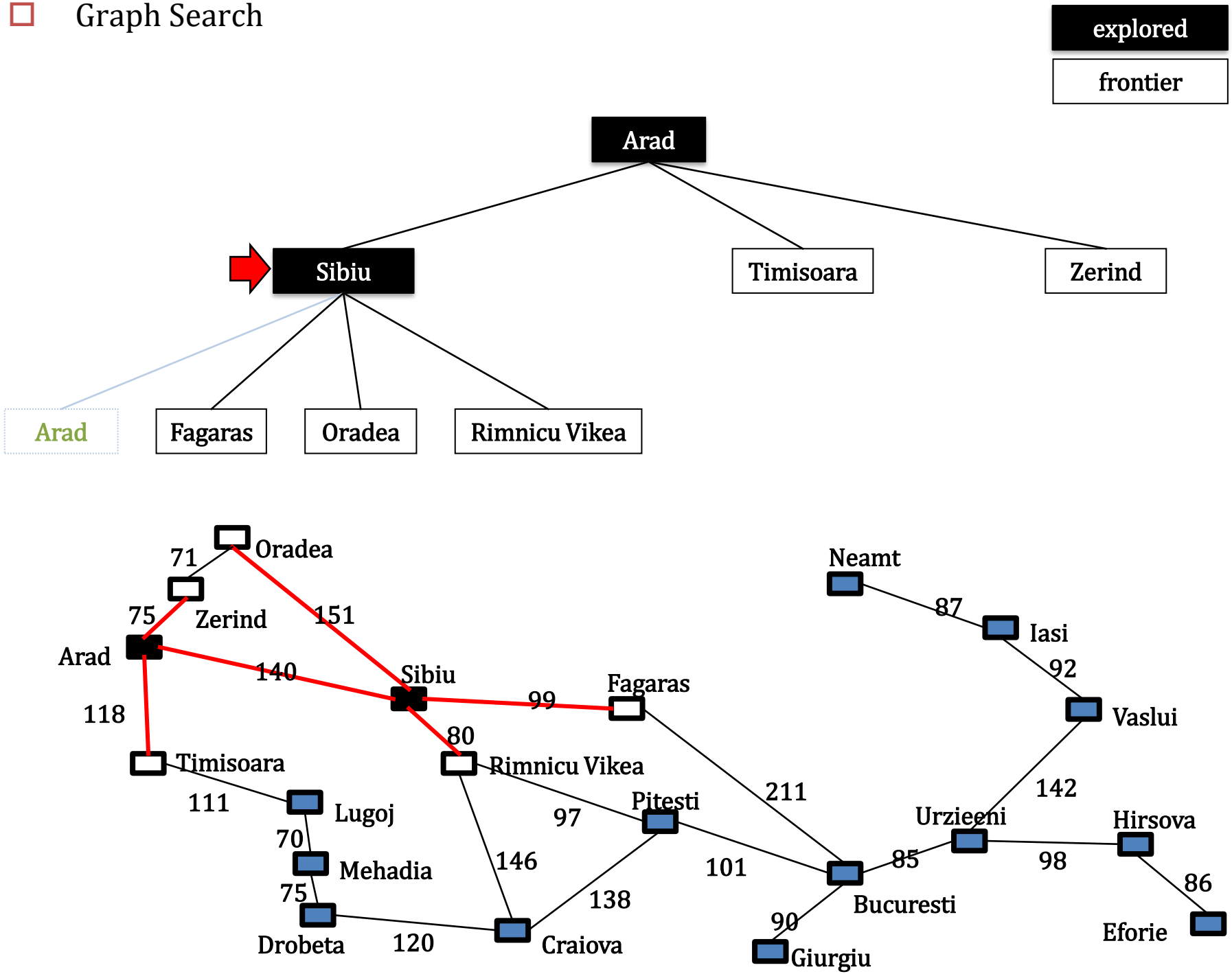
Arad



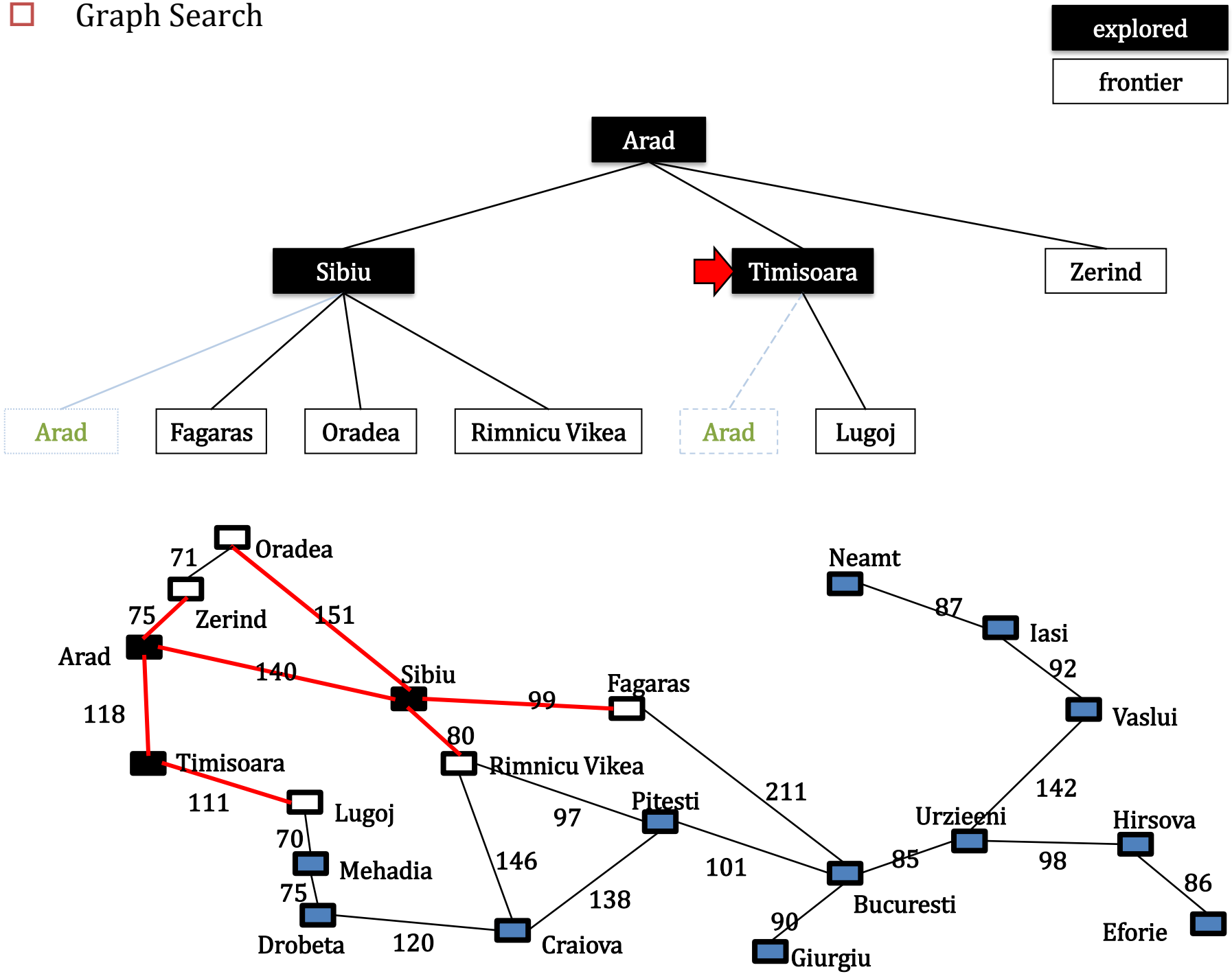
# Graph Search



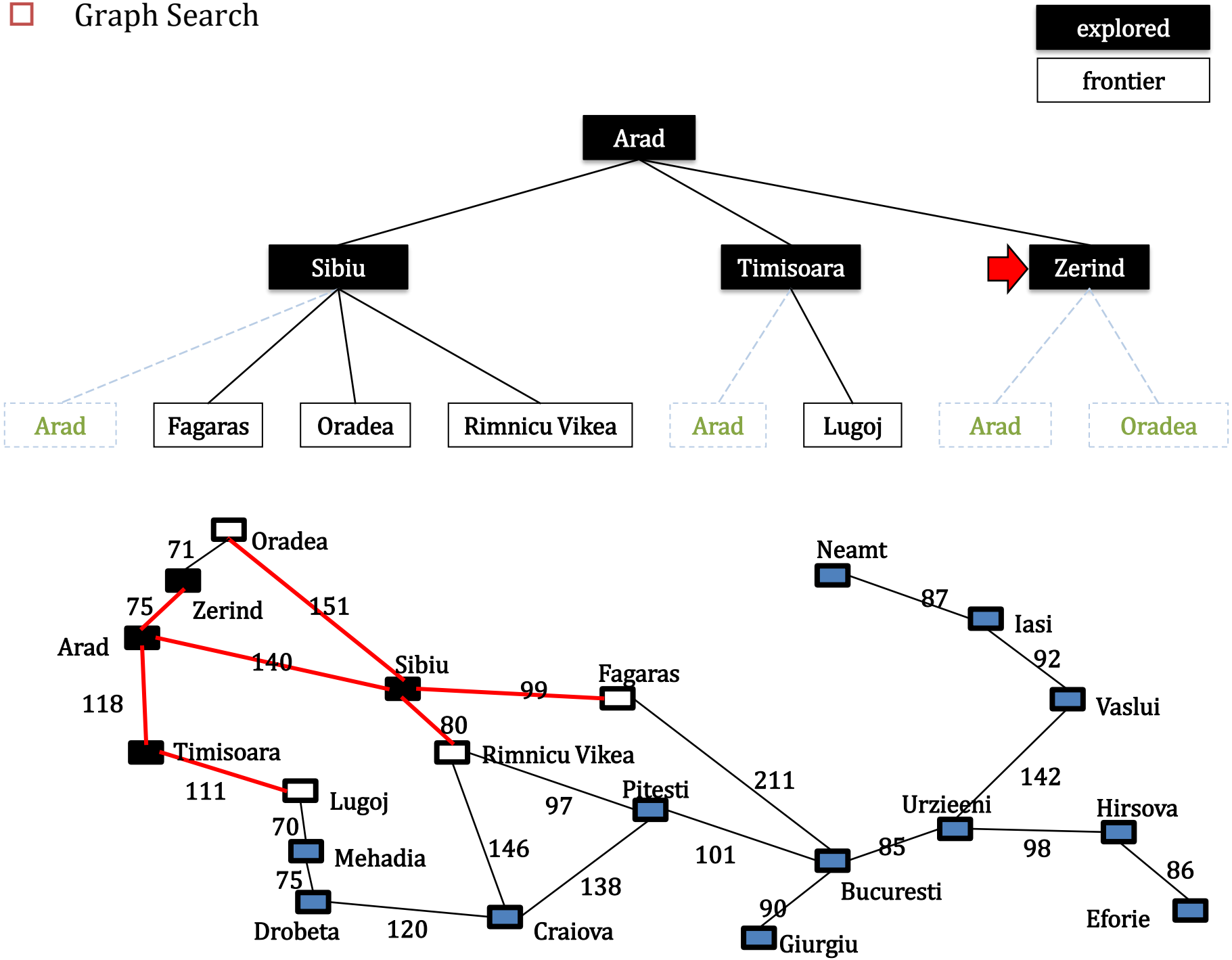
# Graph Search



# Graph Search

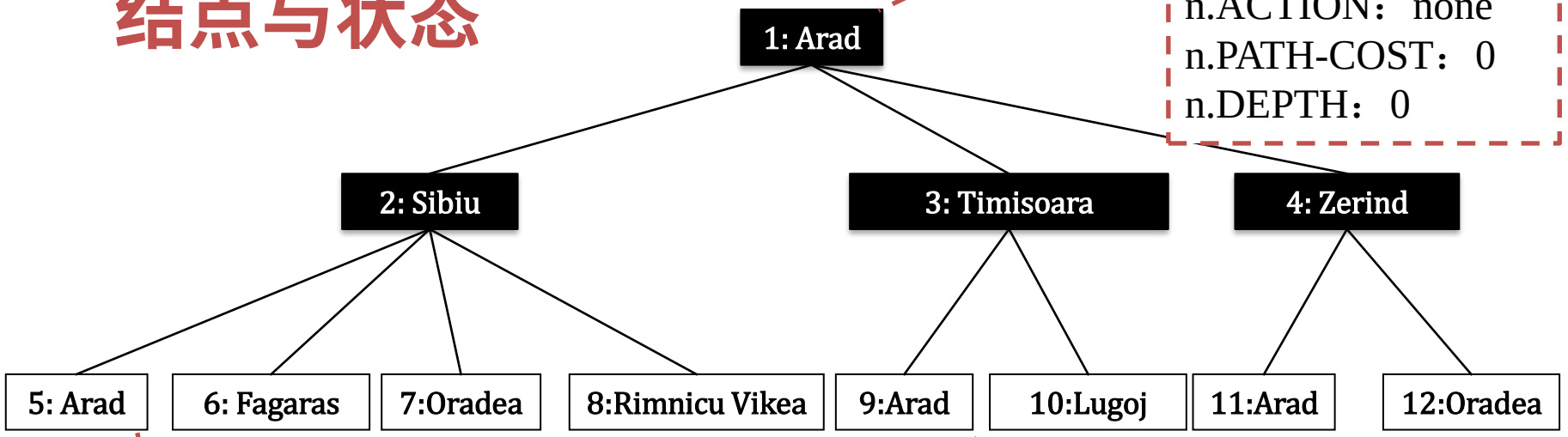


# Graph Search



# (3)树结点的结构

# 结点与状态




n.STATE：Arad
n.PARENT：NULL
n.ACTION：none
n.PATH-COST：0
n.DEPTH：0

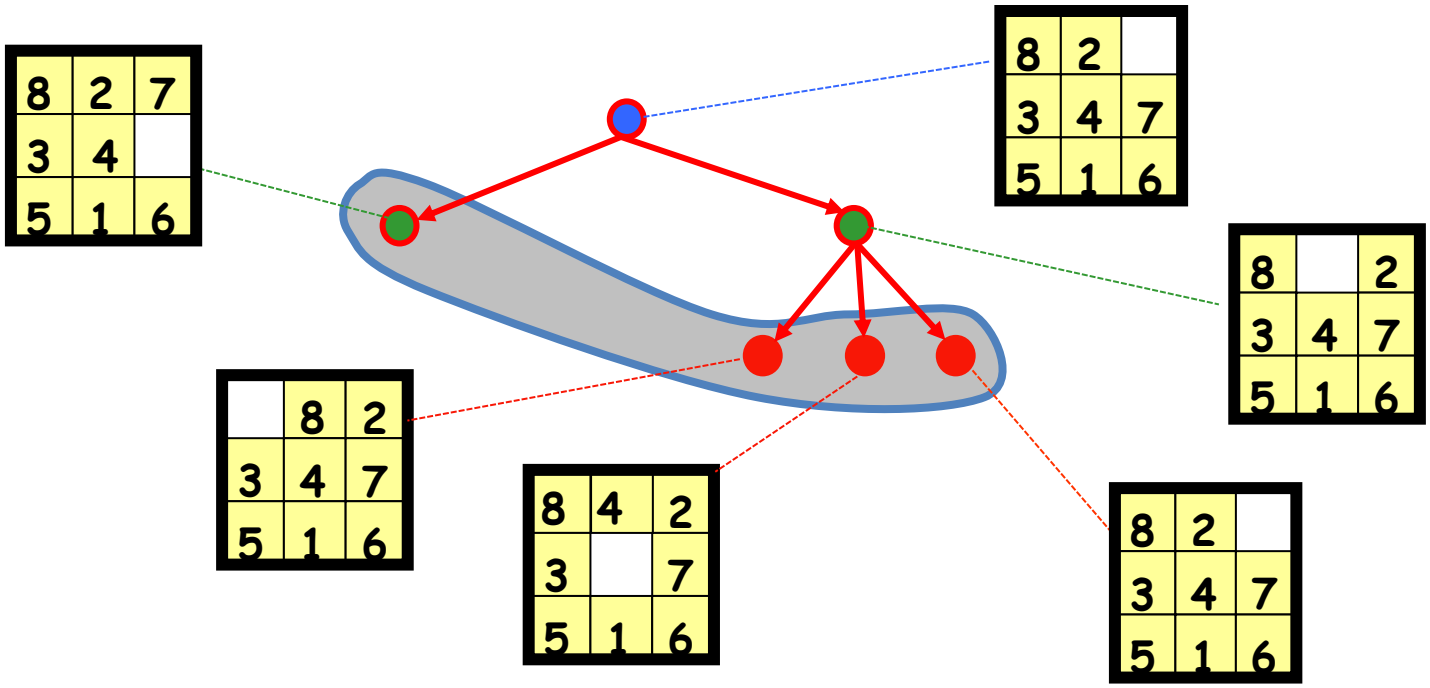
n.STATE：Arad
n.PARENT：2
n.ACTION：Goto(Arad)
n.PATH-COST：280
n.DEPTH：2

n.STATE：Arad
n.PARENT：3
n.ACTION：Goto(Arad)
n.PATH-COST：236
n.DEPTH：2

**function** CHILD-NODE( *problem, parent, action*) **returns** a node
**return** a node with
STATE = *problem*.RESULT( *parent*.STATE, *action*),
PARENT = *parent*, ACTION = *action*,
PATH-COST = *parent*. PATH-COST
+ *problem*.STEP-COST( *parent*.STATE, *action*)

# Frontier vs. Explored

- frontier/边缘结点/待扩展结点
  Explored/已探索结点/已扩展结点
- The frontier is the set of all search nodes that haven't been expanded yet.



# (4)搜索算法的评估

# 完备性、最优性、复杂性

- Completeness(完备性)
  - A search algorithm is complete if it finds a solution whenever one exists
- Optimality(最优性)
  - A search algorithm is optimal if it returns a minimum-cost path whenever a solution exists
- Complexity(复杂性)
  - It measures the time and amount of memory required by the algorithm

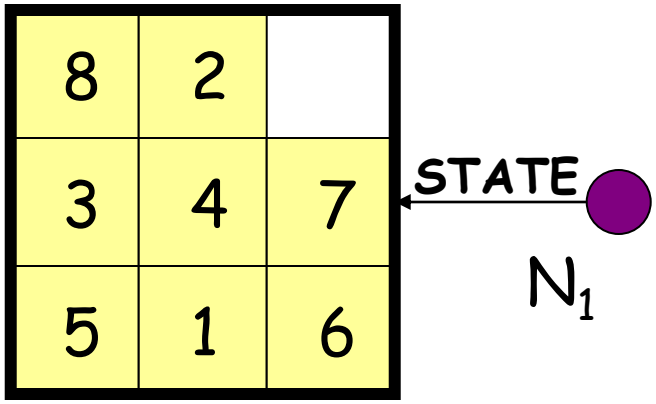
什么决定了搜索策略?

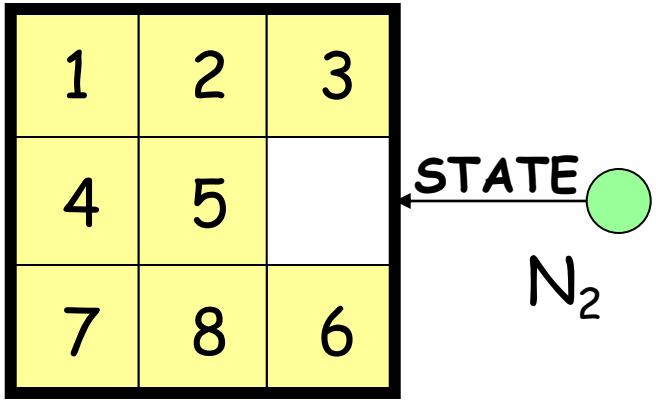
# 3 搜索策略

# 是什么决定了搜索策略?

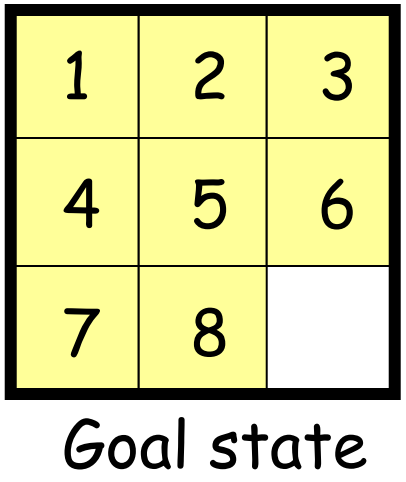
- FRONTIER
  - is the set of all search nodes that haven't been expanded yet. The search algorithm repeatedly chooses and removes a node from the frontier, expand it and insert the resulting nodes to the frontier
  - REMOVE(FRONTIER)
  - INSERT(node, FRONTIER)
- The ordering of the nodes in FRONTIER defines the search strategy

# 盲目搜索不考虑结点离目标有多远




- For a blind strategy, N1 and N2 are just two nodes (at some position in the search tree)



# （1）宽度优先搜索BFS

# 宽度优先搜索BFS(树搜索)

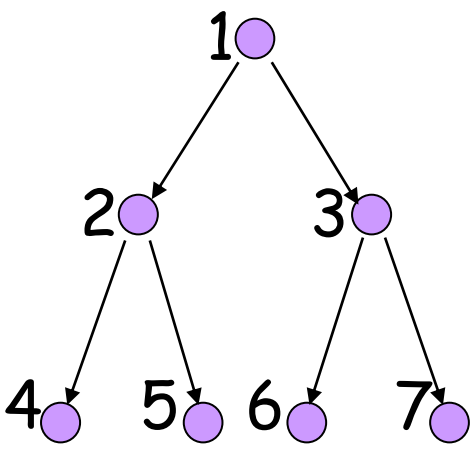
```
function Breadth-First-Search(problem) returns a solution, or failure
    node ← a node with State=problem.Initial-State, Path-Cost=0
    if problem.Goal-Test(node.State) then return Solution(node)
    frontier←{node}

    loop do
        if Empty?(frontier) then return failure
        node ← POP the shallowest node in frontier
        add node.State to explored
        for each action in problem.Actions(node.State) do
            child ← Child-Node(problem, node, action)

            if problem.Goal-Test(child.State) then return Solution(child)
            frontier ← Insert(child, frontier)
```

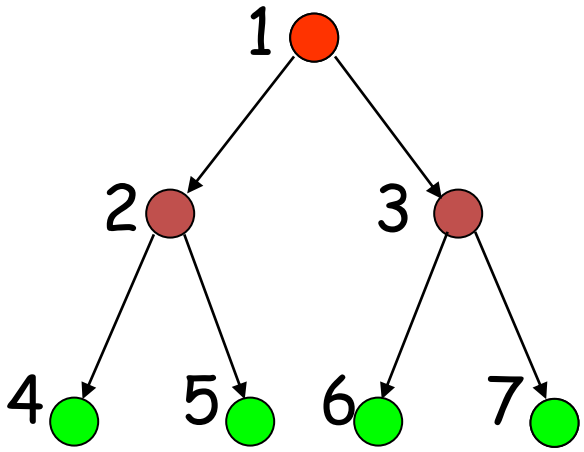
提前进行测试：在加入frontier之前进行测试，为什么？

写出图搜索版本

# BFS示例

- New nodes are inserted at the end of FRONTIER
- and hence the first node is the shallowest node.




FRONTIER = (1)

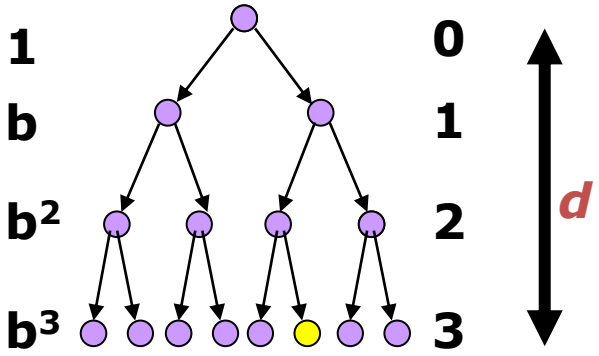
FRONTIER= (2, 3)

FRONTIER = (3, 4, 5)

FRONTIER = (4, 5, 6, 7)

# BFS的完备性、最优性、复杂性

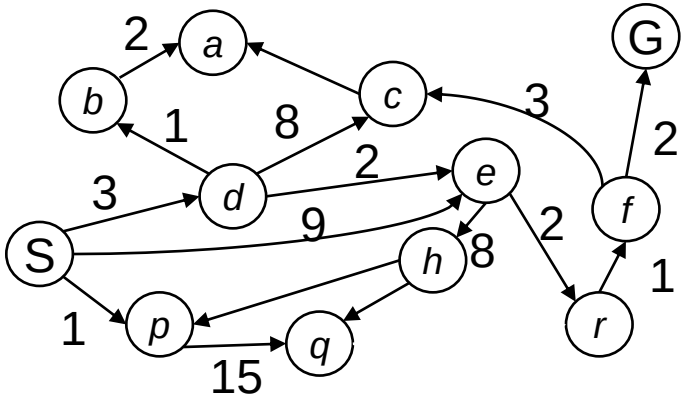
- Branching factor *b* of the search tree
  - Maximum number of successors of any state
  - 假设分支因子b是有限的
- Depth *d* of the shallowest goal node in the search tree
  - Minimal length (≠ cost) of a path between the initial and a goal state
- Breadth-first search is
  - Complete if b is finite
  - Optimal if step cost is 1(单步代价相同)
- Time and space complexity is $1+b+\ldots+b^d=O(b^d)$



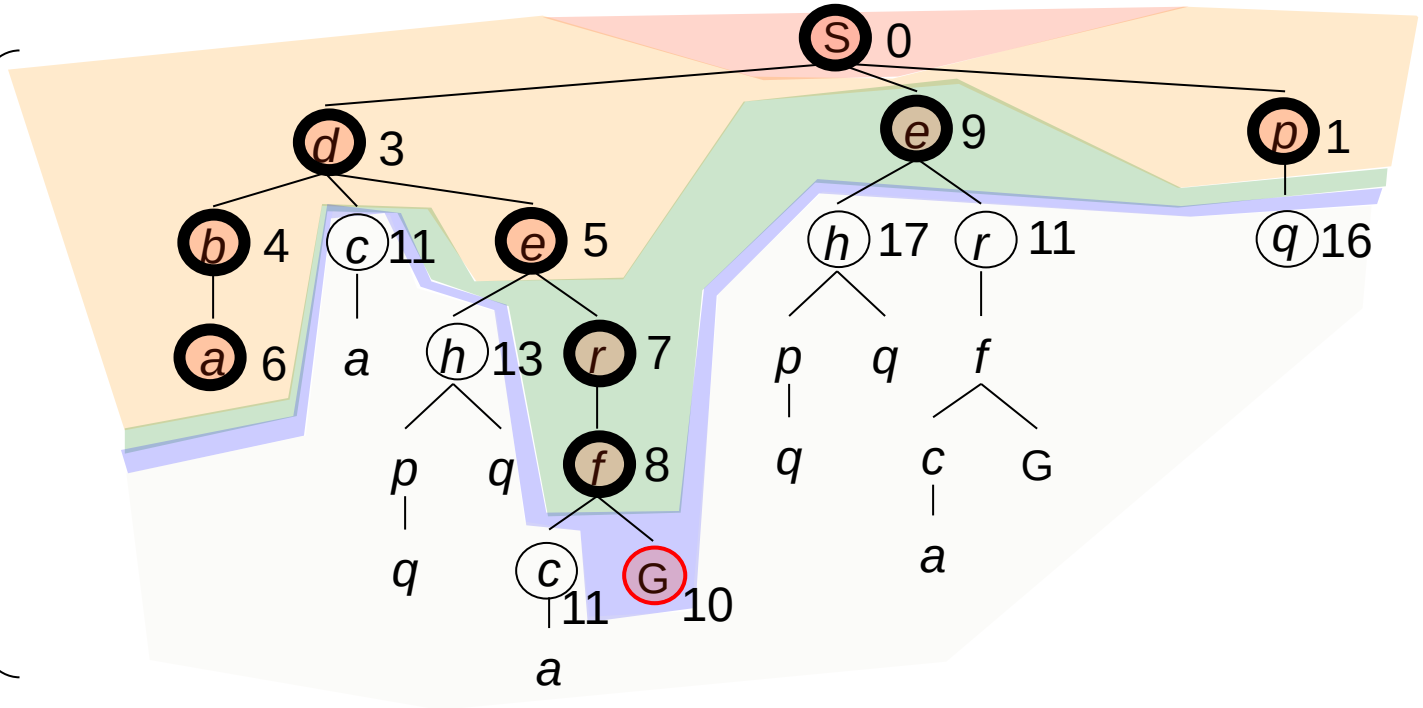
# (2)一致代价搜索UCS

# (2)一致代价搜索UCS(树搜索)

- Expand the cheapest node
- Tree search version:

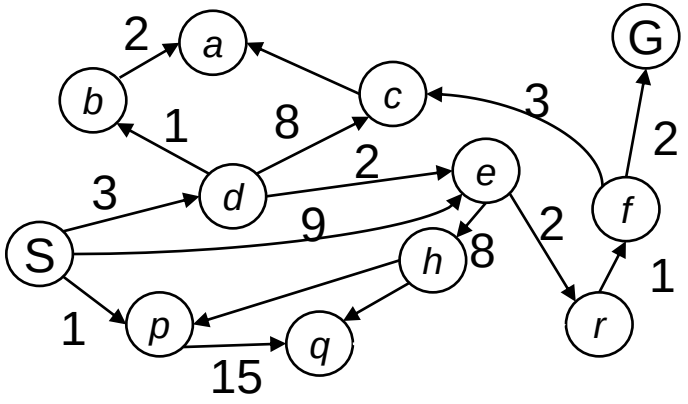



Cost contours

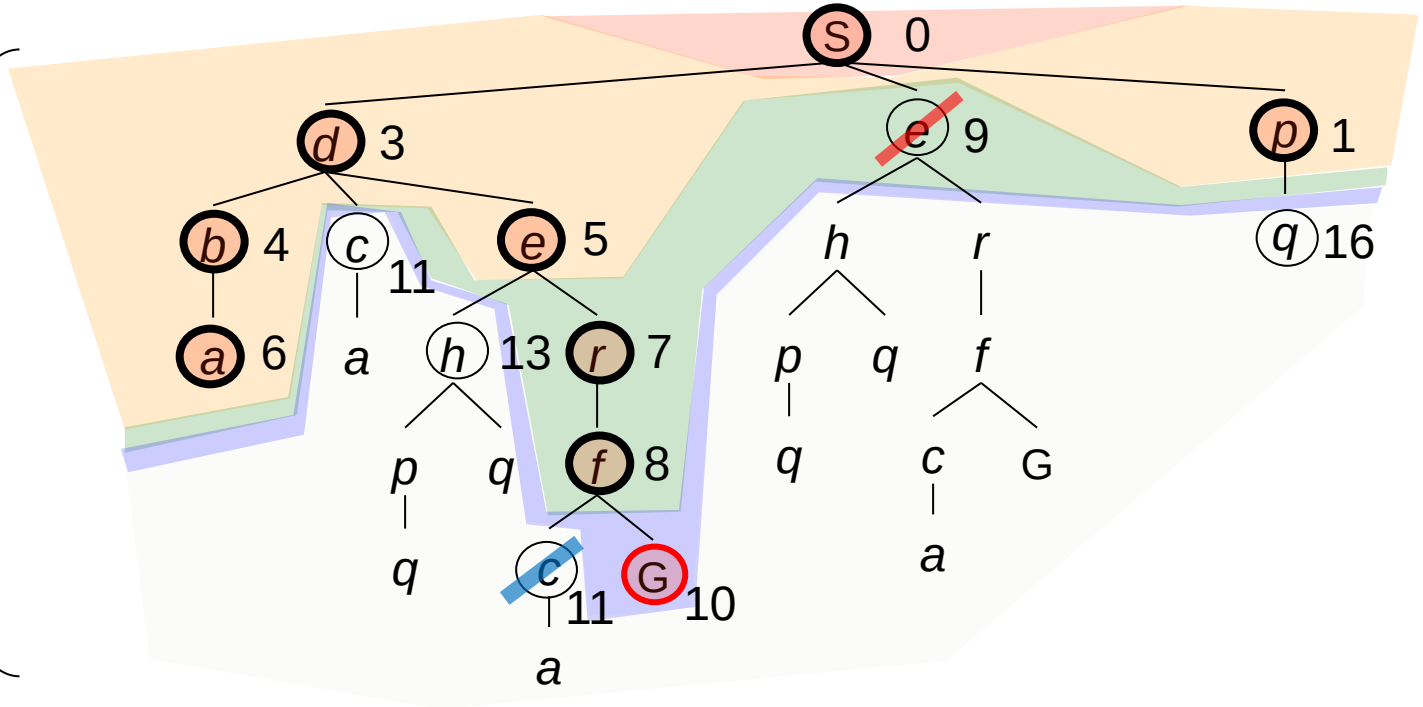


# (2)一致代价搜索UCS(图搜索)

- Expand the cheapest node
- Graph search version




Cost contours



# UCS算法(图搜索)

```
function Uniform-Cost-Search(problem) returns a solution, or failure
  node ← a node with State=problem.Initial-State, Path-Cost=0
  if problem.Goal-Test(node.State) then return Solution(node)
  frontier←{node}
  explored←an empty set
  loop do
    if Empty?(frontier) then return failure
    node ← POP the lowest-cost node in frontier
    if problem.Goal-Test(child.State) then return Solution(child)
    add node.State to explored
    for each action in problem.Actions(node.State) do
      child ← Child-Node(problem, node, action)
      if child.State is not in explored or frontier then
        frontier ← Insert(child, frontier)
      else if child.State is in frontier with higher Path-Cost then
        replace that frontier node with child
      [else delete child]
```

正常测试(不能提前)：从frontier中移出后,先目标测试，然后生成子结点.why?

# UCS的最优性、完备性、复杂性

- It is Optimal
- Complete if the cost of every step is greater than or equal to some small positive constant **ε**. (如果有0代价行动则可能陷入死循环, 例如程序指令的NoOp行动.)
- Time and Space complexity is $O(b^{[c^*/\varepsilon]+1})$
  - where C* is the cost of the optimal solution
  - the depth of the optimal solution is $[c^*/\varepsilon]$
  - $1+b+\ldots+b^{[c^*/\varepsilon]}+b^{[c^*/\varepsilon]+1}= O(b^{[c^*/\varepsilon]+1})$

# Thanks